Chapter 3.

Constants, Variables, Data Type

Data Types

Primary data types

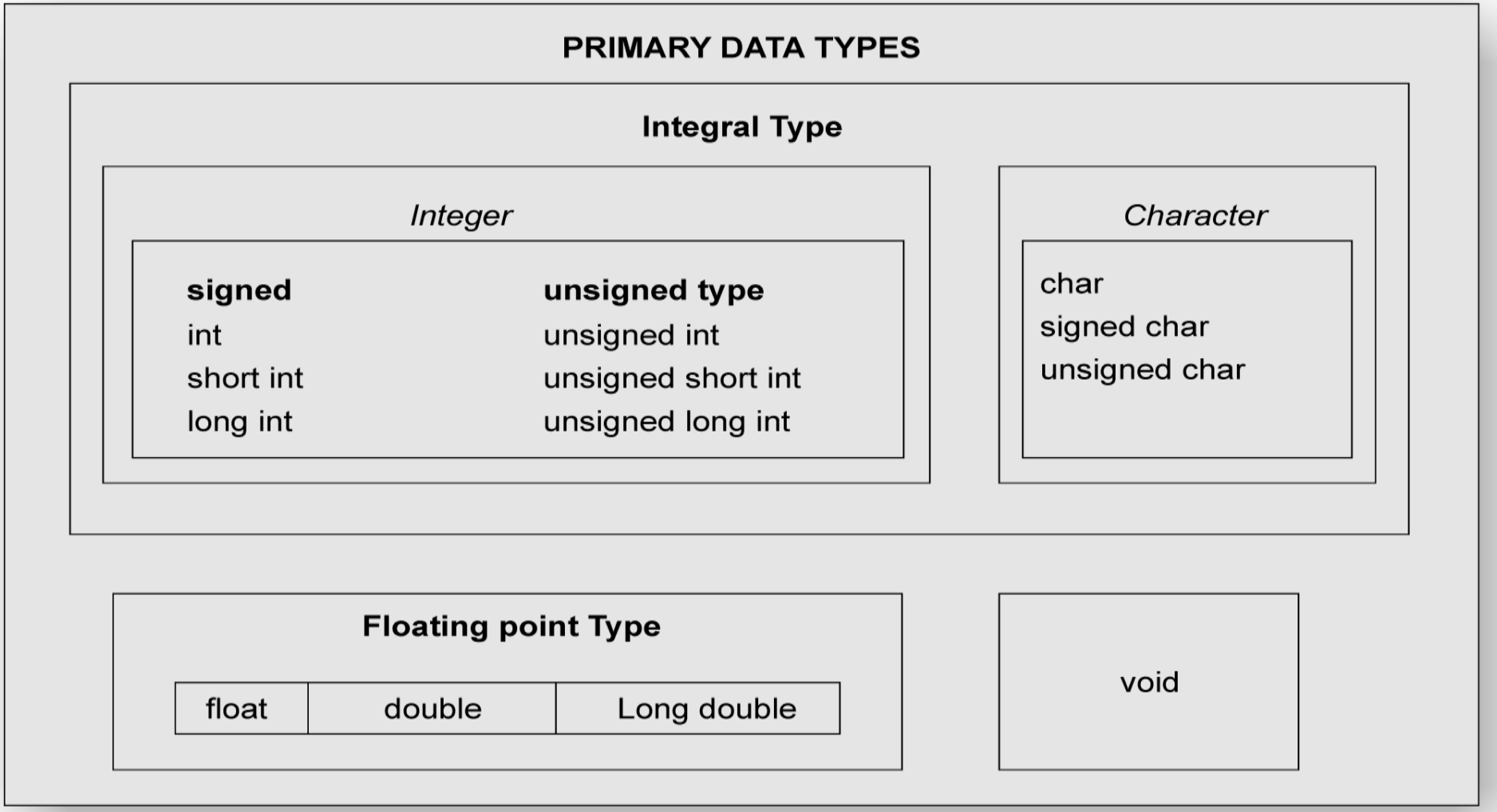


Fig. 2.4 *Primary data types in C*

Data Types

Size and range of basic data types

- Character types on 16-bit machine

Type	Size(bits)	Range
char	8	-128 ~ 127
signed char	8	
unsigned char	8	0 ~ 255

Data Types

Size and range of basic data types

- Integer types on 16-bit machine

Type	Size(bits)	Range
Int or signed int	16	-32,768 ~ 32,767
unsigned int	16	0 ~ 65,536
short int or signed short int	8	-128 ~ 127
unsigned short int	8	0 ~ 255
long int or signed long int	32	-2,147,483 ~ 2,147,483,647
unsigned long int	32	0 ~ 4,294,967,295

Data Types

Size and range of basic data types

- Integer types on 32-bit machine (**widely used**)

Type	Size(bits)	Range
int or signed int	32	-2,147,483,648 ~ 2,147,483,647
unsigned int	32	0 ~ 4,294,967,295
short int or signed short int	16	-32,768 ~ 32,767
unsigned short int	16	0 ~ 65,536
long int or signed long int	32	-2,147,483 ~ 2,147,483,647
unsigned long int	32	0 ~ 4,294,967,295

Data Types

Size and range of basic data types

- Floating point types on 16- / 32- bit machine

Type	Size(bits)	Range
float	32	3.4E-38 ~ 3.4E+38
double	64	1.7E-308 ~ 1.7E+308
long double	80	3.4E-4932 ~ 1.1E+4932

float: 6 digits of precision

double: 14 digits of precision

Declaration of Variables

Primary Type Declaration

Data-type v1, v2, …, vn;

```
float  x, y;
int  code;
short int  count;
long int  amount;
double  deviation;
unsigned  n, m;
char  c, d;
```

Declaration of Variables

Default values of Constants

- **Integer** constants, by default, → **int** type data

Literal	Type	Value
+111	int	111
-222	int	-222
45678U	unsigned int	45,678
-56789L	long int	-56,789
987654UL	unsigned long int	987,654

Declaration of Variables

Default values of Constants

- **Floating point** constants, by default, → **double** type data

Literal	Type	Value
0.	double	0.0
.0	double	0.0
12.0	double	12.0
1.234	double	1.234
-1.2f	float	-1.2
1.23456789	long double	1.23456789

Declaration of Variables

User defined type declaration

typedef ***type*** idenifier;

```
typedef  int  units;
typedef float  marks;

units b1, b2;
marks name1[50], name2[50];
```

Declaration of Variables

Enumerated data type

enum identifier {value1, value2, …, valuen};

0 1 … : assigned by compiler

```
enum day {Mon, Tues, Wed, Thurs, Fri, Sat, Sun};

enum day week_st, week_end;

week_st = Mon;

week_end = Fri;

If(week_st == Tues)

   week_end = Sat;
```

Declaration of Storage Class

Variables location and visibility

```
int m;
main()
{
   int i;   float blc;
   …..              i = 0;
   function1();
 }                  m = i + 3;

function1()
{
   int i;
   float sum;
…       i = 3;
}
```

Global variable: can be seen and used anywhere

Local variables:
only visible and meaningful within the function where they are declared

Declaration of Storage Class

Storage class specifiers

To explicitly declare the scope and lifetime of variables

```
auto int count;
register  char ch;
static int x;
extern long total;
```

Static and **extern** **variables are automatically initialized to zero**

Declaration of Storage Class

Storage classes and their meaning

Storage class	Meaning
auto	Default. Local variable known only to the fution in which it is declared
static	Local variable which exists and retains its value even after the control is transferred to the calling function
extern	Global variable known to all functions in the file
register	Local variable which is stored in the register

```
main()
{
  int i, x;
  for (i=0; i < 5; ++i)
    { x=fsub();
      printf("%d \n", &x);
     }
}

int fsub()
{
  int m = 10;
  static int d = 10;
  d=d+m;
  return(d);
}
```

Assigning Values to Variables

Use of variables

```
value = 0;
year = 2000;
while (year <= 2017)
{
   ….
   value = value + 0.7*income;
   year = year +1;
}
printf(“Amount of savings from year 2000 is %f \n”, value);
```

Assigning Values to Variables

Assignment statements

```
initial_value   = 0;

final_value     = 100;

balance         = 75.84;

yes     = ‘x’;

initial_value = 0; final_value = 100;
```

Assigning Values to Variables

Declaration and Assignment (initialization)

```
int final_value = 100;
char yes = 'x';
p = q = s = 0;
x = y = z = Max;
```

External, static : initialized to zero by default

Automatic : not initialized. Contains garbage

Examples of assignments

Program

```
main()
{
/*..........DECLARATIONS.............................*/
     float     x,p ;
     double    y,q ;
     unsigned  k ;
/*..........DECLARATIONS AND ASSIGNMENTS.............*/
     int       m = 54321 ;
     long int  n = 1234567890 ;
/*..........ASSIGNMENTS.............................*/
     x = 1.234567890000 ;

  y = 9.87654321 ;
  k = 54321 ;
  p = q = 1.0 ;
/*..........PRINTING.................................*/
     printf("m = %d\n", m) ;
     printf("n = %ld\n", n) ;
     printf("x = %.12lf\n", x) ;
     printf("x = %f\n", x) ;
     printf("y = %.12lf\n",y) ;
     printf("y = %lf\n", y) ;
     printf("k = %u p = %f q = %.12lf\n", k, p, q) ;
}
```

Output

```
m = -11215
n = 1234567890
x = 1.234567880630
x = 1.234568
y = 9.876543210000
y = 9.876543
k = 54321 p = 1.000000 q = 1.000000000000
```

%f ➔ 6 decimal places

Assigning Values to Variables

Reading the data from keyboard

scanf("control string", &variable1, &variable2,…);

Address of variable

- Example :

```
int age;
printf("Enter your age: \n");
scanf("%d", &age);
 printf("You are %d years old!\n", age);
```

```
Enter your age:
20
You are  20  years old!
```

scanf("Enter a number %d",&number);

/* Permitted by some compliers*/

Assigning Values to Variables

How to avoid scanf ERROR (C4996)

1. write #pragma warning(disable : 4996)

```
#include <stdio.h>
#pragma warning(disable : 4996)

main()
{
    int age;
        printf("Enter your age: \n");
        scanf("%d", &age);
      printf("You are %d years old!\n", age);
}
```

Assigning Values to Variables

How to avoid Scanf ERROR (C4996)

2. Use scanf_s rather than scanf

```
#include <stdio.h>

main()
{
    int age;
        printf(“Enter your age: \n”);
        scanf_s(“%d”, &age);
      printf(“You are %d years old!\n”, age);
}
```

- But in principle, scanf_s should follow the format below

```
scanf_s("%d", &age, sizeof(int));
```

Program

```
main()
{
      int number;

      printf("Enter an integer number\n");
      scanf ("%d", &number);

      if ( number < 100 )
         printf("Your number is smaller than 100\n\n");
      else
         printf("Your number contains more than two digits\n");
}
```

Type in ‘the number’ and ‘return’ to proceeds to the next statement

Output

```
Enter an integer number
54
Your number is smaller than 100
Enter an integer number
108
Your number contains more than two digits
```

Fig. 2.10 *Interactive investment program*

Program

```
main()
{
      int year, period ;
      float amount, inrate, value ;

      printf("Input amount, interest rate, and period\n\n") ;
      scanf ("%f %f %d", &amount, &inrate, &period) ;
      printf("\n") ;
      year = 1 ;

      while( year <= period )
      {
            value = amount + inrate * amount ;
            printf("%2d Rs %8.2f\n", year, value) ;
            amount = value ;
            year = year + 1 ;
      }
}
```

Output

```
Input amount, interest rate, and period

10000 0.14 5

   1 Rs 11400.00
   2 Rs 12996.00
   3 Rs 14815.44
   4 Rs 16889.60
   5 Rs 19254.15

Input amount, interest rate, and period

20000 0.12 7

   1 Rs 22400.00
   2 Rs 25088.00
   3 Rs 28098.56
   4 Rs 31470.39
   5 Rs 35246.84
   6 Rs 39476.46
   7 Rs 44213.63
```

Defining Symbolic Constants

#define symbolic-name value of constant

#define PI 3.14159

#define MAX 200

NO ‘ ; ’

The constants cannot be assigned any other values later

…

MAX = 120; /* illegal */

Defining Symbolic Constants

- **Understandability**

#define symbolic-name value of constant

```
#define STRENGTH 100
#define PASS_MARK 50
#define MAX 200
#define PI 3.14159
```

Statement	*Validity*	*Remark*
#define X = 2.5	Invalid	'=' sign is not allowed
# define MAX 10	Invalid	No white space between # and define
#define N 25;	Invalid	No semicolon at the end
#define N 5, M 10	Invalid	A statement can define only one name.
#Define ARRAY 11	Invalid	define should be in lowercase letters
#define PRICE$ 100	Invalid	$ symbol is not permitted in name

Table 3.11 *Examples of Invalid #define Statements*

Assigning Values to Variables

Declaring a Variable as Constant

```
const int class_size = 40;     /* cannot be changed */
```

Declaring a Variable as Volatile

```
volatile int date;
```

may be changed at anytime from outside the program

```
volatile const int location = 100;
```

?

Fig. 2.11 *Average of N numbers*

Program

```
#define     N    10                    /* SYMBOLIC CONSTANT */
main()
{
      int     count ;                      /* DECLARATION OF */
      float   sum, average, number ;       /* VARIABLES */
      sum     = 0 ;                        /* INITIALIZATION */
      count   = 0 ;                        /* OF VARIABLES */
      while( count < N )
      {
            scanf("%f", &number) ;
            sum = sum + number ;
            count = count + 1 ;
      }
      average = sum/N ;
      printf("N = %d Sum = %f", N, sum);
      printf(" Average = %f", average);
}
```

Output

```
1
2.3
4.67
1.42
7
3.67
4.08
2.2
4.25
8.21
N = 10    Sum = 38.799999 Average = 3.880000
```

Fig. 2.12 *Temperature conversion—fahrenheit-celsius*

Program

```
#define F_LOW      0          /* ----------------- */
#define F_MAX    250          /* SYMBOLIC CONSTANTS */
#define STEP      25          /* -----------------*/

main()
{
      typedef float REAL ;           /* TYPE DEFINITION */
      REAL fahrenheit, celsius ;     /* DECLARATION */

      fahrenheit = F_LOW ;           /* INITIALIZATION */
      printf("Fahrenheit Celsius\n\n") ;
      while( fahrenheit <= F_MAX )
      {
            celsius = ( fahrenheit - 32.0 ) / 1.8 ;


            printf(" %5.1f %7.2f\n", fahrenheit, celsius);
            fahrenheit = fahrenheit + STEP ;
      }
}
```

Output

```
Fahrenheit      Celsius

     0.0          -17.78
    25.0           -3.89
    50.0           10.00
    75.0           23.89
   100.0           37.78
   125.0           51.67
   150.0           65.56
   175.0           79.44
   200.0           93.33
   225.0          107.22
   250.0          121.11
```

Common Programming Errors

Three kinds of errors :

1. Syntax Error

- A violation of the C grammar rules, detected during program translation (compilation).
- Statements cannot be translated and program cannot be executed

Debugging:

Find errors and correct them

Common Programming Errors

Three kinds of errors :

2. Run-time errors

- An attempt to perform an invalid operation, detected during program execution
- Occurs when computer performs an illegal operation, such as dividing a number by zero
- The computer will stop executing the program, and displays a diagnostic message indicates the line where the error was detected

Common Programming Errors

Three kinds of errors :

3. Logic Error/Design Error

- An error caused by following an incorrect algorithm
- It does not cause run-time error – no error messages
- The only sign of logic error – incorrect program output
- Can be detected by testing the program thoroughly, comparing its output to calculated results
- **To prevent** – ***carefully check the algorithm before you actually type program codes***